

	Departamento: Departamento de Ciencias de la Computación.
	Carrera: Software Asignatura: Análisis y Diseño de Software NRC: 30724
	Apellidos y nombres del estudiante: • Lasso Sebastián • Valencia Yuliana • Villarroel Justin Taller N°1

ANTECEDENTES

En el desarrollo de software existen diferentes formas de organizar un sistema para que sea más fácil de entender, mantener y mejorar. Una de estas formas es mediante el uso de patrones de diseño, los cuales ayudan a resolver problemas comunes de programación.

Dentro de estos patrones se encuentran los patrones estructurales, que sirven para organizar las relaciones entre clases y objetos de una mejor manera. En este trabajo se estudiarán los patrones Composite, Proxy y Bridge, ya que cada uno cumple una función diferente dentro de un sistema.

El patrón Composite ayuda a trabajar con objetos individuales y grupos de objetos de la misma manera. El patrón Proxy permite controlar el acceso a ciertas partes del sistema usando un intermediario. Finalmente, el patrón Bridge ayuda a separar una parte del sistema de su implementación para que sea más flexible y fácil de modificar.

Estos patrones pueden aplicarse en un sistema de matriculación estudiantil para mejorar la organización del código y facilitar procesos como el registro y administración de estudiantes.

OBJETIVO

Analizar e implementar los patrones de diseño estructurales **Composite, Proxy y Bridge** en el desarrollo de un sistema de matriculación estudiantil, con el fin de mejorar la organización, acceso y flexibilidad de la estructura del software.

DESARROLLO

1. Composite

Descripción:

El patrón Composite pertenece a los patrones de diseño estructurales, ya que se enfoca en la forma en que las clases y objetos se organizan dentro de un sistema para construir estructuras complejas.

Este patrón permite componer objetos en estructuras jerárquicas tipo árbol, de manera que los objetos individuales y los grupos de objetos puedan ser tratados de forma uniforme. En otras palabras, un objeto principal puede estar formado por varios objetos secundarios, manteniendo una misma interfaz de operación.

La principal ventaja del patrón Composite es que simplifica la gestión de estructuras complejas, ya que el sistema puede manipular tanto elementos simples como compuestos utilizando los mismos métodos.

Como funciona:

El patrón Composite funciona organizando objetos en forma de una estructura jerárquica, donde un objeto principal puede contener otros objetos. Esto permite trabajar de la misma manera con un solo elemento o con un grupo de elementos. Por ejemplo, en un sistema de matriculación, se puede manejar un estudiante individual o una lista de estudiantes usando las mismas operaciones del sistema.

2. Proxy (Apoderado)

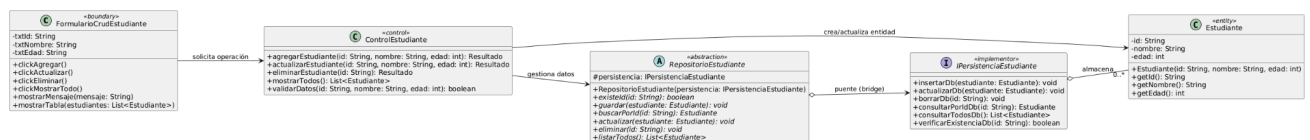
Descripción: Es un patrón de diseño estructural que proporciona un sustituto o marcador de posición (placeholder) para otro objeto con el fin de controlar el acceso a éste.

Cómo funciona: El Proxy implementa exactamente la misma interfaz que el objeto real al que representa (Sujeto Real). Cuando el cliente hace una solicitud, la hace al Proxy creyendo que es el objeto original. El Proxy intercepta esta solicitud y realiza operaciones adicionales de gestión antes o después de pasar la solicitud al objeto real.

3. Bridge (Puentes)

Descripción: Es un patrón de diseño estructural que desacopla una abstracción de su implementación, de modo que ambas puedan variar o evolucionar de forma completamente independiente.

Cómo funciona: Normalmente, si tienes una clase con muchas variantes (por ejemplo, Forma que puede ser Circulo o Cuadrado) y necesitas que funcione en múltiples plataformas (por ejemplo, Windows o Linux), podrías terminar usando herencia múltiple o creando una cantidad explosiva de subclases (CirculoWindows, CirculoLinux, CuadradoWindows, etc.). El patrón Bridge resuelve esto cambiando la herencia por composición.



CONCLUSIONES

RECOMENDACIONES

Sangolquí, a 09 de abril de 2026